Reading Report: SGD for Large-Scale Machine Learning and Sparse Data

Student ID: 26125082 Course: CS161

October 17, 2026

Proposed topic: Implementing Stochastic Gradient Descent (SGD) in C++ and comparing its weight-optimization techniques with scikit-learn's `SGDRegressor`.

---

1 Background and Objectives

SGD is a foundational algorithm for large-scale machine learning. In Python, a user only needs to call `SGDRegressor` to obtain an already optimized implementation. This report reviews two classic papers on making SGD run fast, extracts their core techniques, and then asks: **when SGD is re-implemented from scratch in C++, do these techniques retain their value, and how does the C++ version compare with scikit-learn?**

2 Literature Reviewed

#	Reference	Rationale for selection
1	L. Bottou (2012), *Stochastic Gradient Descent Tricks*, in *Neural Networks: Tricks of the Trade* (2nd ed.)	Explains why SGD suits large datasets; presents the $w = s\,v$ implementation trick and the step-size schedule for linear models with L_2 regularization
2	S. Shalev-Shwartz, Y. Singer, N. Srebro, A. Cotter (2011), *Pegasos: Primal Estimated sub-GrAdient SOlver for SVM*, *Mathematical Programming*	Uses the same sparse representation trick $w = a\,v$, the step-size schedule $1/(\lambda t)$ that requires no tuning, and a convergence analysis for the L_2-regularized problem

Both papers center on linear models with L_2 regularization, which is exactly the configuration that this project will implement and compare.

3 Summary of Each Paper

3.1 Bottou (2012): SGD Tricks

Idea. Each step uses the gradient of a single randomly chosen example: $w \leftarrow w - \gamma_t \nabla Q(z_t, w)$.

Why SGD suits large-scale data. The total error decomposes into three terms: approximation error, estimation error, and optimization error. When *computation time* is the bottleneck, solving the optimization problem to high accuracy is wasteful. SGD is the "worst" optimization algorithm in terms of convergence rate, yet it reaches a given level of expected risk in the asymptotically shortest time.

Practical recommendations. Shuffle the data, normalize the inputs, monitor both the training cost and the validation error, verify gradients with finite differences, and tune the learning rate on a small subsample.

Linear models with L_2 regularization (Section 5).

- Represent the weights as $w = s\,v$, where s is a scalar (denoted W in the original paper), so that each step costs time proportional to the number of nonzero entries of the example: the shrinkage step due to regularization reduces to multiplying s by a constant.
- Step-size schedule $\gamma_t = \gamma_0\,(1 + \gamma_0 \lambda t)^{-1}$.
- ASGD (weight averaging) yields near-optimal results after roughly one epoch on some problems.

Experiments (RCV1, SVM). SGD runs in 1.4 s, compared with 66 s for SVMPerf and 23,642 s for SVMLight, with comparable test error ($\approx 6.02\%$).

3.2 Pegasos (Shalev-Shwartz et al., 2011)

Algorithm. Pegasos directly solves the primal SVM problem $\frac{\lambda}{2}\|w\|^2$+hinge loss using stochastic sub-gradient descent with step size $\eta_t = 1/(\lambda t)$, so there is no step-size parameter to tune.

Theory. The method needs $\tilde{O}(1/(\lambda\varepsilon))$ iterations (previously $\Omega(1/\varepsilon^2)$). With a linear kernel, the running time is $\tilde{O}(d/(\lambda\varepsilon))$, where d is the number of nonzero features per example, **independent of the number of training examples**. The analysis relies on the objective being λ-strongly convex.

Extensions. Mini-batches, kernels, the sparse representation $w = a\,v$, and other losses (log-loss, ε-insensitive loss for regression, multiclass, etc.), provided the loss is convex with bounded subgradients.

Experiments.

- CCAT: Pegasos 0.16 s versus SVM-Perf 3.6 s.
- Running time depends on λ rather than on the number of examples; a small λ slows the method down.
- Sampling without replacement is faster than sampling with replacement.
- A fixed step size in the style of Zhang is very sensitive to η.

4 Takeaways from the Literature

1. **The efficiency of SGD lies in the update rule, not in the implementation language.** Both papers use the same trick: maintaining $w = s\,v$ so that each step costs $O(\mathrm{nnz}(x))$ rather than $O(n)$, while the L_2 shrinkage step costs only $O(1)$.
2. **The step-size schedule is where the two papers complement each other.** Bottou proposes $\gamma_0(1+\gamma_0\lambda t)^{-1}$ together with selecting γ_0 on a subsample; Pegasos uses $1/(\lambda t)$ and avoids tuning altogether. The two schedules coincide (up to a shift of one in the iteration index) when $\gamma_0 = 1/\lambda$.
3. **The empirical evidence comes from classification on sparse data** (SVM on RCV1 and CCAT), not from regression.

4. **In Python, these techniques are already packaged.** The scikit-learn documentation describes the `'optimal'` schedule as following Bottou's heuristic; whether scikit-learn actually uses the $w = s\,v$ representation must be verified against the library's Cython source (the `WeightVector` class and `_sgd_fast`) rather than assumed.
5. This leads to the question addressed by the project: if SGD is re-implemented in C++, do these optimizations remain as effective?

5 Research Questions

Main question. Scikit-learn already optimizes the weight update in SGD. When SGD is re-implemented in C++, do these optimizations retain their effectiveness (speed and convergence quality), and how does the C++ version compare with `SGDRegressor`?

RQ1. Does the $w = s\,v$ optimization remain effective in C++?

As the density of nonzero entries in the data varies, how does the per-iteration time of the C++ implementation using $w = s\,v$ differ from that of a C++ implementation that updates the entire w directly, and how do both differ from `SGDRegressor`?

Hypothesis. The benefit stems from the per-step complexity (reducing $O(n)$ to $O(\text{nnz}(x))$), not from the language, and therefore persists in C++:

- On sparse data, the $w = s\,v$ version is markedly faster, and the gap widens with dimensionality.
- On dense data ($\text{nnz} \approx n$), both versions are of the same order $O(n)$, differing only by a constant factor.
- The direct-update C++ version slows down as dimensionality grows, even though it is compiled.

Validation.

- Three configurations: direct-update C++, $w = s\,v$ C++, and `SGDRegressor`, with the same sample order and the same step-size schedule.
- Varied factors: density of nonzero entries, dimensionality, and number of samples.
- Confirm that the two C++ versions produce the same solution (differences at the level of machine precision).
- Monitor the phenomenon $s \to 0$ and the rescaling mechanism. Note that with $\eta_t = 1/(\lambda t)$ we have $1 - \eta_1 \lambda = 0$ at the first step, so s collapses to zero immediately and this case requires separate handling.

RQ2. Is the convergence quality equivalent to scikit-learn's?

With the same loss, L_2 regularization, and step-size schedule, does the C++ implementation achieve the same MSE, RMSE, and R^2, and the same closeness to the closed-form Ridge solution, as `SGDRegressor`?

Factors to investigate.

- Step-size schedules: $\gamma_0(1 + \gamma_0 \lambda t)^{-1}$ (Bottou), $1/(\lambda t)$ (Pegasos), `'optimal'`, and scikit-learn's default schedule.
- Selecting γ_0 on a subsample, as recommended by Bottou.
- Whether or not the data are standardized.

Validation.

- Measure MSE, RMSE, and R^2 on the test set, and the distance $\|w - w_{\text{ridge}}\|$ to the closed-form Ridge solution.
- When comparing with Ridge, the regularization coefficient must be converted so that the objectives match: Ridge uses the *sum* of squared errors, whereas SGD uses the *mean* over samples, so for the loss $\frac{1}{2}(\hat{y} - y)^2$ we have $\alpha_{\text{Ridge}} \approx m\,\alpha_{\text{SGD}}$, where m is the number of samples. The intercept must also be treated identically on both sides.
- Use the same sample order, random seed, and number of epochs for all configurations.

6 Tentative Plan

The experimental plan follows one principle: **each research question uses the type of data best suited to it.** RQ1 concerns the cost of a single update as a function of density, dimensionality, and sample size, so it requires synthetic data in which each of these quantities can be varied independently. RQ2 concerns convergence quality, so it requires real data small enough to compare against the closed-form Ridge solution. Within each group of experiments, the configurations under comparison share the same data, sample order, and hyperparameters, so that any measured difference can be attributed solely to the implementation.

Notation: n is the number of dimensions, m the number of samples, and k the number of nonzero entries per row.

6.1 Data Preparation

RQ1: sparse synthetic data.

1. The data are generated in Python (seed 42) and written to a binary file in CSR format (`indptr`, `indices`, `data`, `y`; types float64/int32). C++ and Python read the same file, so no rounding discrepancies arise from conversion through text.
2. For each row, k distinct positions are chosen at random, the values are drawn from $\mathcal{N}(0, 1)$, and the row is normalized to $\|x\|_2 = 1$. As a result, the product $\eta\|x\|^2$ is constant as k varies, and the same η does not cause divergence at high densities.
3. The true weights are $w^* \sim \mathcal{N}(0, 1)$ (dense); the labels are $y = x \cdot w^* + \epsilon$ with $\epsilon \sim \mathcal{N}(0, 0.1^2)$.
4. The data are not centered, since centering would make the sparse matrix dense and eliminate precisely the property under study.

Three experimental grids are used, each varying a single factor:

Grid	Varied factor	Held fixed
A	dimensionality $n \in \{10^3, 10^4, 10^5, 10^6\}$	$k = 20$, $m = 10\,000$
B	$k \in \{5, 50, 500, 2500, 5000\}$, i.e., density from 0.1% to 100%	$n = 5000$, $m = 5000$
C	number of samples $m \in \{10^3, 10^4, 10^5, 10^6\}$	$n = 10^4$, $k = 20$

RQ2: three housing-price datasets.

Dataset	Size (samples × features)	Target variable	Source
Real Estate Valuation (Taiwan)	414×6	house price per unit area	UCI, id 477
California Housing	$20\,640 \times 8$	median house value	`sklearn.datasets.fetch_california_housing`
King County House Sales	$21\,613 \times \sim 17$ numeric columns	`price`	Kaggle `kc_house_data` or OpenML id 42092

- The three datasets differ in size (from a few hundred to over twenty thousand samples) and in number of features (6 to about 17), so conclusions about correctness depend less on any single dataset.
- For California Housing, the scikit-learn version is preferred. The `housing.csv` file circulating online may contain missing values in the `total_bedrooms` column (this must be re-checked before the data source is finalized).
- All three datasets are dense and low-dimensional, so RQ2 does not aim to demonstrate the benefit of the $w = s\,v$ representation. The purpose of RQ2 is to verify the correctness and performance of the C++ implementation.

Minimal preprocessing.

- Only numeric columns are used; for King County, `id`, `date`, and `zipcode` are dropped (`zipcode` is a categorical code).
- No imputation and no outlier removal. Natural outliers, such as the strongly right-skewed prices in King County or the capped prices in California, are retained.
- The train/test split is 80/20 with seed 42; the split indices are saved so that Python and C++ share them.
- X is standardized with `StandardScaler` fitted on the training set. y is standardized using the training-set mean and standard deviation to stabilize SGD, and converted back to the original units when computing metrics.
- `assert` statements are added before each run to check that all values are finite and that there are no NaNs.
- Because outliers remain, both RMSE and MAE are reported (MAE is less dominated by outliers).

6.2 Training Configuration and Equivalence with scikit-learn

RQ1. All three implementations use the squared loss, L_2 regularization with $\alpha = 10^{-4}$, `fit_intercept=False`, the `invscaling` schedule ($\eta_0 = 0.01$, `power_t` $= 0.25$), 3 epochs, and the same fixed sample order.

RQ2. The `SGDRegressor` configuration is: `loss='squared_error'`, `penalty='l2'`, `alpha`, `tol=None`, `max_iter=E`, `early_stopping=False`, `average=False`, `shuffle=False`, initialization $w = 0$, and the `invscaling` schedule (the default of `SGDRegressor`). The number of epochs E is shared across models. The closed-form Ridge solution serves as an (optional) reference, with the regularization coefficient converted as $\alpha_{\text{Ridge}} \approx m\,\alpha_{\text{SGD}}$, as stated in RQ2.

To make the C++ implementation equivalent to scikit-learn, each update step is implemented in the following order:

1. Compute the prediction $p = w \cdot x + b$ using w before shrinkage.
2. Shrink the weights due to regularization: $w \leftarrow w \cdot \max(0,\, 1 - \eta\alpha)$.
3. Apply the gradient update: $w \leftarrow w - \eta\,(p - y)\,x$.

The remaining details are as follows.

- The gradient is clipped at $\pm 10^{12}$, and the shrinkage factor s is reset when it falls below 10^{-9}, as in scikit-learn.
- The `invscaling` schedule: $\eta_t = \eta_0 / t^{0.25}$, with t starting at 1 and counted continuously across epochs.
- The intercept is not regularized and is updated as $b \leftarrow b + \text{update} \cdot \text{intercept_decay}$. The value of `intercept_decay` (expected to be 1 for dense data and 0.01 for sparse data) must be checked against the `_sgd_fast` source code before implementation. RQ1 uses `fit_intercept=False` and is therefore unaffected.
- The `double` type is used throughout. The goal is not bit-for-bit agreement, but to measure the deviation of the coefficients after 1 epoch and after E epochs, and to set an acceptance threshold based on pilot runs. The scikit-learn version is stated explicitly in the report.

6.3 Measurement Methodology

- **C++ timing:** `steady_clock` around the training loop (excluding I/O); compiled with `-O2`, without `-ffast-math`, single-threaded; one warm-up run followed by 7 timed runs, reporting the median and IQR.
- **scikit-learn timing:** since `fit` has a fixed overhead, $T(E)$ is measured for $E \in \{1, \ldots, 5\}$ with `tol=None`, and the slope is taken as the time per epoch. The data are passed in advance as CSR float64.
- **Unit:** μs per update = time of one epoch $/\, m$.
- **RQ2:** report the total `fit` time of scikit-learn together with the pure loop time of the C++ implementation. On small datasets, scikit-learn is slower mainly because of overhead rather than the algorithm; this will be noted in the report.

6.4 Results to Be Reported

RQ1: three figures, each with three curves (direct-update C++, $w = s\,v$ C++, and `SGDRegressor`):

Figure	x-axis	y-axis	Expectation
1 (grid A)	dimensionality n (log)	μs/update (log)	The direct-update version grows proportionally to n; the $w = s\,v$ version and scikit-learn remain nearly flat (dependent on k)
2 (grid B)	density k/n (%)	μs/update	The three curves converge as the density approaches 100%
3 (grid C)	number of samples m (log)	μs/update	Flat curves, showing that the per-step cost is independent of m

An accompanying supplementary table reports: the speedup ratio $t_{\text{direct}}/t_{w=s\,v}$; the deviation $\|w_{s\,v} - w_{\text{direct}}\|/\|w\|$ together with the final loss (to demonstrate that the two C++ versions yield the same solution); and the number of resets of the shrinkage factor s.

RQ2: a metrics table for each dataset, covering the closed-form Ridge model (reference, optional), `SGDRegressor`, and C++ (using $w = s\,v$, emulating scikit-learn), with the columns: RMSE, MAE, R^2, $\|w - w_{\text{ridge}}\|/\|w_{\text{ridge}}\|$, training time (ms), and number of epochs. This is accompanied by a check that $|\text{RMSE}_{\text{C++}} - \text{RMSE}_{\text{sklearn}}|$ falls below a threshold; the initially proposed threshold is 10^{-6} and will be calibrated after pilot runs.

6.5 Risks and Additional Experiments

1. **The `optimal` schedule risks divergence under the squared loss.** scikit-learn's heuristic sets typw $= \sqrt{1/\sqrt{\alpha}}$ and $\eta_0 = \text{typw}/\max(1, \text{dloss})$. With $\alpha = 10^{-4}$, a one-step probe on scikit-learn 1.9.1 gives $\eta_0 = 10$, whereas $\|x\|^2$ of the standardized data is approximately the number of features (6 to 17). Consequently $\eta\|x\|^2$ far exceeds the commonly used stability threshold for least-squares updates ($\eta\|x\|^2 < 2$) during the early steps. For this reason the primary configuration is `invscaling`; the `optimal` schedule is run as a secondary experiment and its outcome is recorded even if it diverges. This is also where Bottou's step-size schedule will be discussed within RQ2.
2. **The rescaling mechanism may not be triggered in RQ1.** With $\eta_0 = 0.01$, $\alpha = 10^{-4}$, and 3 epochs, the shrinkage factor s only decreases to about 0.91 at the largest scale ($m = 10^6$; a rough estimate), so the reset threshold of 10^{-9} is very likely never reached. If this mechanism needs to be verified, a supplementary configuration with a larger α will be added.
3. **Possible extensions (if time permits):** Bottou's schedule $\gamma_0(1+\gamma_0\lambda t)^{-1}$ and Pegasos's schedule $1/(\lambda t)$, as well as runs without data standardization, corresponding to the factors listed in RQ2. The $1/(\lambda t)$ schedule in particular requires special handling of the first step because $1 - \eta_1\lambda = 0$.

References

[1] Bottou, L. (2012). Stochastic Gradient Descent Tricks. In *Neural Networks: Tricks of the Trade* (2nd ed.), LNCS 7700, Springer.

[2] Shalev-Shwartz, S., Singer, Y., Srebro, N., Cotter, A. (2011). Pegasos: Primal Estimated sub-GrAdient SOlver for SVM. *Mathematical Programming.*

[3] scikit-learn documentation: `sklearn.linear_model.SGDRegressor`.

AI Usage Declaration

In preparing this report, the following AI tools were used:

- **Gemini Flash 3.8 (medium)** was used to generate ideas for the datasets, to design the figures and tables of results, and to draft the tentative experimental plan.
- **Claude Sonnet 5.5** was used to draft and edit the text of the document and to generate the PDF file.

The author reviewed the AI-assisted output and remains responsible for the final content of this report.